



Année Universitaire 2025-2026

RAPPORT DE STAGE TER

Gendered News : refonte du système

GETALP



Présenté par

Rachel Peretti

Aylin Guventurk

Jury

Master MIASHS : M. Pellier Damien

Master MIASHS : Mme. Landry Aurélie

LIG : M. Portet François

Remerciements

Nous souhaitons adresser nos remerciements les plus sincères aux personnes qui nous ont accompagnées et soutenues tout au long de ce Travail d'Études et Recherches, ainsi qu'à l'élaboration de ce rapport.

Nous remercions l'équipe GETALP pour leur accueil et leur bienveillance. Nous exprimons notre gratitude à François Portet, Ange Richard et Patrick Juen pour leur aide et leurs conseils durant cette expérience.

Nous tenons à remercier l'équipe enseignante du Master MIASHS, notamment Damien Pellier pour les conseils donnés dans le cadre du TER.

Enfin, nous remercions également Hervé Blanchon pour l'aide à la recherche des missions pour notre TER.

Introduction	4
Contexte et état initial du projet	5
Présentation de l'équipe GETALP	5
Présentation de Gendered News	5
Problématiques identifiées	6
Organisation et démarche de travail	7
Méthodologie adoptée	7
Outils et pratiques professionnelles	7
Plan de développement	8
Réalisations techniques	8
Refonte de l'architecture	8
Réorganisation du projet	9
Tests unitaires	9
Documentation technique	10
Installation du projet	10
Documentation du code : génération automatique avec pdoc	11
Analyse critique du travail réalisé	11
Difficultés rencontrées	11
Améliorations possibles	12
Perspectives d'évolution du projet	13
Conclusion	13
Glossaire	15
Bibliographie	16
[1] Gendered News Dashboard :	
https://genderednews.univ-grenoble-alpes.fr/dashe/	16
[2] Pdoc Documentation : https://pdoc.dev/	16
Résumé	17
Annexe	18
Figure 1 - Diagramme de l'ancienne architecture logicielle	18
Figure 2 - Diagramme de la nouvelle architecture logicielle	19
Figure 3 - Flux des étapes de la récupération d'articles	20
Figure 4 - Diagramme de Gantt	21

Introduction

Ce rapport présente le travail réalisé dans le cadre du projet Travail d'Études et de Recherche (TER) pour la première année du Master Mathématiques et Informatique appliquées aux Sciences Humaines et Sociales. Ce projet a pour objectif de mettre en pratique les connaissances acquises au cours de la formation dans un contexte proche du monde professionnel, tout en prenant du recul sur les choix techniques, méthodologiques et organisationnels effectués. Un résumé synthétique des travaux réalisés est présenté en fin de ce document afin d'en fournir une vue d'ensemble.

Ce document est organisé de la façon suivante: une première partie présente la structure d'accueil pour le TER, ainsi que le contexte de Gendered News. Ensuite sont décrites l'organisation et les méthodes de travail appliquées durant le projet. Une quatrième partie traite des réalisations techniques, notamment la refonte de l'architecture logicielle* et la génération de documentation. Une cinquième partie est consacrée à l'analyse critique de notre travail, accompagnée des perspectives d'évolution du projet. En conclusion, ce rapport se termine par un bilan de stage et met en évidence les compétences professionnelles et techniques développées au cours du TER.

Contexte et état initial du projet

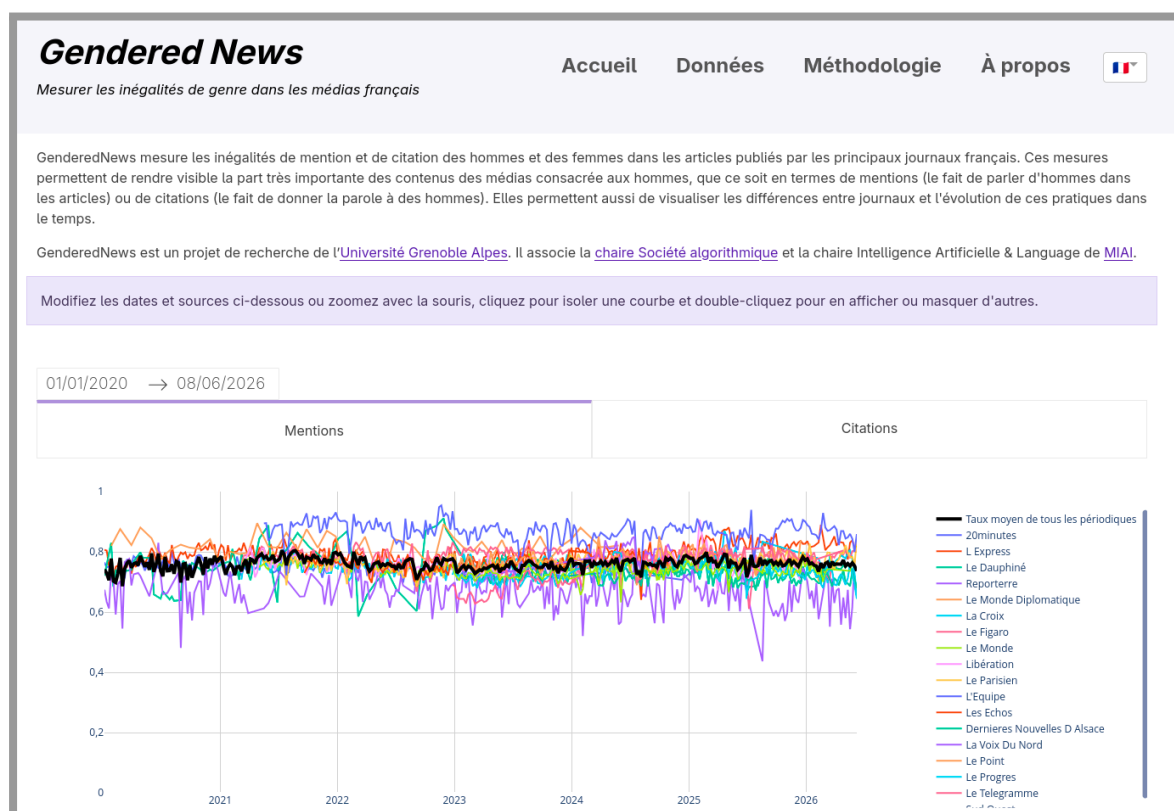
Présentation de l'équipe GETALP

Le travail d'étude et de recherche se déroule au sein du LIG (Laboratoire d'Informatique de Grenoble) regroupant des équipes de chercheurs, avec 22 équipes pour 5 axes de recherche. L'équipe GETALP (Groupe d'Étude en Traduction Automatique/Traitement Automatisé des Langues et de la Parole), créée en 2007, en fait partie et étudie le traitement du langage écrit/oral, avec plusieurs sous-axes de recherche: la traduction et transmission automatique de la parole, le traitement et l'analyse d'interactions, ou encore le processus automatique de clarification du sens. GETALP collabore avec d'autres équipes du LIG, mais aussi de nombreux laboratoires grenoblois ou également à l'international avec un important réseau de contacts, avec par exemple des ambassadeurs en Malaisie ou aux États-Unis.

Le projet est initié par le laboratoire Pacte, mais le TER s'effectue dans l'équipe GETALP et est encadré par François Portet en collaboration avec Ange Richard, doctorante-ingénieure d'études et Patrick Juen, développeur principal.

Présentation de Gendered News

Le projet Gendered News^[1], en ligne depuis 2021, est un ensemble d'outils numériques, notamment un site web, qui mettent en évidence les inégalités de représentation entre les hommes et les femmes dans les principaux journaux de presse française en ligne. Le projet fait des études sur des journaux connus comme *Le Monde*, *Libération* ou encore *L'Equipe*.



Ici nous avons un exemple d'un des graphiques présents sur le site web. Ce sont des représentations graphiques qui permettent de visualiser clairement la part des hommes mentionnés et cités dans les articles, et la variation de cette part selon les différents sujets comme le sport, l'éducation, la mode. Sur une période donnée, ici une période allant de janvier 2020 à janvier 2026, on obtient le taux de mentions d'hommes dans une liste de journaux définis.

À terme, cette plateforme a pour but d'être en partenariat avec des agences de presse comme l'AFP.

Le projet présente un historique de développement important, caractérisé par l'intervention successive de nombreux développeurs et par l'accumulation de choix de technologies variées, dont certaines ont depuis été remplacées ou abandonnées. Le site a également eu plusieurs itérations avant l'actuelle.

Problématiques identifiées

Au début du projet, nous avons identifié plusieurs problématiques. La première concerne la maintenance des scrapers*: des outils permettant de récupérer le contenu d'une page web. Les sites de presse évoluent régulièrement, tant au niveau de leur structure que de leurs mécanismes de protection contre le scraping automatisé, et ces évolutions peuvent provoquer des erreurs d'extraction ou la récupération de données incomplètes, ce qui peut rendre certains scrapers* inopérants.

Une seconde problématique concerne la dette technique* accumulée au fil des années, c'est-à-dire l'accumulation de problèmes et lacunes dans le projet dû à son historique et aux différents développeurs ayant contribué. Certaines dépendances utilisées par le projet sont devenues obsolètes et présentent des problèmes de compatibilité avec les versions récentes de leur écosystème.

Enfin, l'architecture du projet souffrait d'un manque d'homogénéité hérité de son historique de développement. Les fichiers de configuration étaient répartis dans différents répertoires sans organisation clairement définie, ce qui rendait la compréhension du code et sa maintenance plus complexes. Par ailleurs, les tests existants n'étaient plus fonctionnels, limitant la capacité à détecter les régressions. Dans ce contexte, l'un des objectifs du projet était de réaliser une refonte de l'architecture logicielle*, c'est-à-dire de clarifier les responsabilités des différents modules et d'améliorer la maintenabilité globale du système.

Organisation et démarche de travail

Méthodologie adoptée

Dans le cadre de ce projet, nous avons mis en place des routines de travail pour la réalisation des missions; en mettant l'accent sur l'organisation de points quotidiens pour se tenir à jour des tâches en cours. À la fin de chaque semaine, nous organisons également une rétrospective en binôme afin d'analyser le travail accompli, d'évaluer l'atteinte des objectifs fixés et de planifier les tâches de la semaine suivante; ce que nous écrivions dans un compte-rendu qui était ensuite envoyé à l'équipe.

L'ensemble des documents de travail, notes et comptes rendus était centralisé sur un espace partagé Google Drive. Pour le suivi opérationnel des tâches, nous utilisons Trello selon une approche de type Kanban, permettant de visualiser les travaux à réaliser, en cours et terminés. En début de projet, nous avons également élaboré un diagramme de Gantt (voir [Figure 4 - Diagramme de Gantt](#)) afin d'estimer la charge de travail et de planifier les différentes étapes du TER.

Au niveau de l'équipe Gendered News, des réunions appelées "Point projet" étaient organisées chaque mois. Ces points permettaient de présenter l'avancement des différents travaux, d'échanger sur les problématiques rencontrées et de suivre les activités menées par les autres membres du projet. Certaines réunions étaient également consacrées à des sujets plus transversaux, tels que les partenariats ou les perspectives d'évolution de la plateforme. Nous réalisons aussi des points individuels avec Patrick Juen et Ange Richard pour poser nos questions et bénéficier de revues de code.

Outils et pratiques professionnelles

Afin d'assurer un développement structuré et maintenable, nous avons mis en œuvre plusieurs outils et pratiques professionnelles au cours du projet.

La gestion du code source reposait sur Git*, permettant de suivre l'historique des modifications, de collaborer efficacement et de sécuriser les développements grâce à un système de versionnement. Le projet Gendered News est hébergé sur un dépôt Gitlab, et nous avons fait en sorte de travailler sur des branches dédiées avant leur intégration au projet principal.

Nous avons porté une attention particulière à la documentation du projet; les modules développés ont été documentés afin d'améliorer leur compréhension et leur maintenabilité. Nous reviendrons plus en détail sur cet aspect dans la suite du document.

Enfin, plusieurs pratiques favorisant la qualité logicielle ont été appliquées. Les développements ont fait l'objet de revues de code régulières avec les encadrants du projet, permettant d'identifier des pistes d'amélioration et de valider les choix techniques réalisés.

Plan de développement

Dans cette partie, nous évoquons les principaux travaux réalisés au cours du projet ainsi que les choix techniques qui ont guidé leur mise en œuvre.

Réalisations techniques

Refonte de l'architecture

Une part importante du projet a d'abord été d'analyser l'architecture existante afin d'en comprendre le fonctionnement et d'identifier les principaux points de blocage. Cette phase d'étude s'est révélée particulièrement importante compte tenu de l'historique du projet et du nombre de contributeurs ayant participé à son développement au fil des années.

Initialement, le projet comportait cette organisation : [Figure 1 - Diagramme de l'ancienne architecture logicielle*](#).

Notre analyse de l'organisation a mis en évidence plusieurs difficultés :

- Notions de classe modèles et logique métiers non distinctes : les responsabilités des différents modules ne sont pas clairement définies. Certaines classes mélangent à la fois la représentation des données et la logique métier, ce qui complexifie leur compréhension et leur maintenance.
- Organisation de l'architecture logicielle* : plusieurs fonctionnalités sont réparties dans différents répertoires sans structure clairement identifiable
- Fichiers et composants non utilisés.
- Notions obsolètes : nous constatons la présence de fonctionnalités devenues obsolètes, telles que certains modules liés à Twitter.
- Manque de documentation technique permettant de comprendre rapidement l'architecture du projet.
- Tests dysfonctionnels : les tests existants ne sont plus fonctionnels, limitant la capacité à détecter les régressions lors des évolutions du code.

Ces constats nous ont amenées à engager une réflexion sur la refonte des modules liés au scraping* qui est un processus automatisé permettant de récupérer le contenu de pages web, au parsing* qui est l'étape consistant à analyser des données brutes afin d'en extraire des informations exploitables par un programme, et au traitement des articles. Dans le cadre de ce projet, nous désignons par processing* l'ensemble des traitements appliqués aux articles après leur extraction, tels que les calculs des indicateurs de représentation des genres ou encore leur catégorisation.

Nous pouvons voir la nouvelle architecture dans cette figure en annexe : [Figure 2 - Diagramme de la nouvelle architecture logicielle*](#). L'intérêt est de surtout mieux responsabiliser les rôles des modules. Dans l'ancienne version, les rôles ne semblaient pas clairs et beaucoup de logique était éparpillée de manière désordonnée. Cette nouvelle architecture repose sur une meilleure séparation des responsabilités.

Dans la nouvelle version, le flux des étapes est découpé comme montré dans la figure suivante : [Figure 3 - Flux des étapes de la récupération d'articles](#).

La première étape consiste à récupérer les flux RSS des différents périodiques afin d'obtenir les URLs des articles à traiter. Un flux RSS est un format de diffusion de contenu permettant à un site web de fournir les métadonnées à jour sur un contenu publié. Dans le cadre du projet Gendered News, les flux RSS sont utilisés comme point d'entrée du processus de collecte. Ces URLs sont ensuite transmises à `ParserFactory`, chargé de sélectionner automatiquement le `parser*` adapté en fonction du domaine concerné. Par exemple, une URL provenant du site *lemonde.fr* sera traitée par le composant *LeMondeParser*. Chaque `parser*` est responsable de l'extraction des informations d'un article (titre, auteurs, date de publication, contenu, etc.) et retourne un objet *Article*. Une fois le contenu parsé et les informations d'un article récupéré, le service `ArticleService` se charge de prendre ces *Article*, et d'appliquer les `Processing`. Un service est un composant dont le rôle est de porter la logique métier et d'orchestrer les différentes opérations nécessaires à une fonctionnalité de l'application. Les articles enrichis sont ensuite enregistrés en base de données.

Cette nouvelle organisation apporte plusieurs améliorations par rapport à l'architecture précédente. La séparation des responsabilités rend le code plus lisible et plus simple à maintenir. L'ajout d'un nouveau périodique est également simplifié puisqu'il suffit principalement d'implémenter un nouveau parseur respectant l'interface commune sans modifier le reste du système.

Une fois cette conception finalisée, nous l'avons implémentée sur le système, en retirant le code non utilisé, afin de partir d'une base plus claire.

Réorganisation du projet

Une fois cette conception finalisée, nous avons entrepris son implémentation au sein du système. Cela a impliqué un important travail de nettoyage du code existant, comprenant la suppression des modules inutilisés, la réorganisation des composants et la clarification des responsabilités de chaque élément.

Nous avons également ajouté un dossier dédié aux scripts de débogage et aux tests manuels. Par ailleurs, un dossier consacré à la documentation a été créé pour centraliser les ressources techniques du projet et encourager les futurs contributeurs à documenter leurs développements.

En tout, la branche originale comportait 534 fichiers, et notre branche dérivée en comporte à présent 380.

Tests unitaires

Lors de l'analyse du projet existant, nous avons constaté que les tests unitaires présents n'étaient plus fonctionnels. Cette situation rendait difficile la validation des modifications apportées au code et augmentait le risque de régression lors des développements. Nous avons alors développé des tests pour les modules `RssSource`, `ParserFactory`, `ArticleParser`, `Article` et `Processing`.

Dans une démarche d'amélioration continue de la qualité logicielle, nous avons également mis à jour la configuration de l'intégration continue* à l'aide de GitLab CI. À chaque modification du code source, les tests unitaires sont maintenant exécutés automatiquement afin de détecter rapidement d'éventuelles régressions.

Documentation technique

Installation du projet

Le projet est hébergé sur GitLab et doit d'abord être récupéré sur la machine cible à l'aide de Git*.

Après le clonage du dépôt, un environnement virtuel Python (venv) doit être créé manuellement par l'utilisateur, ce qui isole les dépendances du projet du reste du système d'exploitation afin de ne pas créer de potentiels conflits avec d'autres projets et leurs dépendances. Une fois créé, l'environnement est activé avant toute exécution.

Les dépendances du projet sont ensuite installées à partir du fichier `requirements.txt`. Ce fichier centralise toutes les bibliothèques Python nécessaires au fonctionnement de l'application, facilitant l'installation du projet sur une nouvelle machine.

Le projet a été développé et testé sous Ubuntu 22.04 LTS, c'est donc l'environnement recommandé pour l'exécuter.

L'automatisation du déploiement et de la mise à jour des données repose sur un script nommé `cron_start`, exécuté chaque matin par le service Cron; mais l'utilisateur peut choisir l'heure de son choix. Ce script assure l'ensemble des opérations nécessaires au fonctionnement du site web.

Il active d'abord l'environnement Python puis lance les traitements principaux du projet. Il exécute ensuite les scripts de calcul permettant la catégorisation des articles avec le modèle FLAUBERT. Une fois ces calculs terminés, le script charge les variables d'environnement nécessaires à l'application et génère l'ensemble des graphiques utilisés par l'interface web.

Le site web est lancé automatiquement via un serveur Flask en arrière-plan, afin de s'assurer que les données affichées soient régulièrement à jour et que l'application reste accessible sans intervention manuelle de l'utilisateur.

Afin d'éviter tout conflit avec une instance déjà en cours d'exécution, le script vérifie également si un processus Flask est déjà présent. Si un processus est détecté, celui-ci est arrêté avant le redémarrage de l'application. Cela est notamment utile si le script s'était déjà activé auparavant.

Nous avons nettoyé ce script en retirant des lignes appelant des environnements obsolètes, et avons adapté les lignes existantes à notre nouvelle architecture.

Documentation du code : génération automatique avec pdoc

Nous avons mis en place un outil qui conduit à enrichir la documentation directement dans le code, où les principales classes et leurs responsabilités ont été décrites, ainsi que pour les méthodes.

Pour cela, Pdoc[\[2\]](#) est utilisé dans le projet. Cette bibliothèque fonctionne en analysant directement les modules Python du projet. Lors de son exécution, l'outil importe les fichiers spécifiés, et récupère les commentaires rédigés dans les docstrings. Il génère ensuite des pages HTML structurées présentant l'architecture.

Dans notre cas, nous avons créé un script permettant de générer la documentation des principaux modules du projet. À chaque mise à jour du code ou de sa documentation, il suffit de relancer ce script pour produire une version actualisée des pages HTML. Cette approche garantit la cohérence entre le code source et la documentation et de conserver celle-ci dans le système de versionnement du projet, conformément aux recommandations de l'équipe Gendered News.

Plusieurs outils existent pour générer de la documentation Python, notamment Pdoc, Sphinx ou encore MKDocs. Le choix s'est porté sur Pdoc car il permet de produire automatiquement une documentation HTML à partir des commentaires originaux du code source, sans nécessiter une configuration importante, ni la rédaction de fichiers de documentation supplémentaires. Pdoc est une bibliothèque largement utilisée dans les projets Python, reconnue pour sa stabilité, ce qui explique pourquoi nous avons préféré l'intégrer dans Gendered News.

Analyse critique du travail réalisé

Tout au long du projet, nous avons cherché à adopter une démarche professionnelle fondée sur la planification des tâches, la documentation des développements et des échanges réguliers avec les encadrants. Néanmoins, plusieurs difficultés sont apparues au cours du TER. Certaines étaient liées à l'état technique du projet existant, tandis que d'autres relevaient davantage de contraintes organisationnelles.

Difficultés rencontrées

La principale difficulté rencontrée au cours de ce projet est liée à l'état initial de ce dernier. Le projet Gendered News possède plusieurs années d'historique et a été modifié par de nombreux contributeurs successifs, ce qui a conduit à l'accumulation d'une dette technique* importante, rendant la compréhension du fonctionnement global particulièrement complexe. Une part importante du temps du projet a ainsi été consacrée à l'analyse du code existant.

Cette phase d'appropriation s'est révélée plus longue que prévu. En conséquence, certaines améliorations initialement envisagées n'ont pas pu être menées à leur terme. Par exemple, nous aurions souhaité consacrer davantage de temps à la remise en état de certains scrapers* défectueux, à l'amélioration des tests ou encore l'optimisation des parcours en base de données.

Nous avons également constaté plusieurs limitations liées à l'environnement technique du projet. Certaines ressources matérielles et logicielles n'étaient pas toujours adaptées aux besoins actuels du système, en particulier les traitements appliqués aux articles présentent des temps d'exécution importants, pouvant atteindre entre quarante minutes et une heure pour certaines opérations. Cette problématique a d'ailleurs été discutée lors des dernières réunions du projet, car elle constitue un frein potentiel à son évolution et à son exploitation future. Notre intervention a souligné la nécessité d'une réflexion plus globale sur l'infrastructure et les ressources allouées au projet.

Au-delà des aspects techniques, plusieurs difficultés organisationnelles ont également été rencontrées. La première concerne la transmission des connaissances autour du projet. Malgré la disponibilité des encadrants pour répondre à nos questions, la complexité du projet et l'absence d'une documentation suffisamment centralisée ont entraîné une phase d'apprentissage relativement longue. Avec davantage de documentation technique ou une formation initiale plus structurée, une partie de ce temps aurait probablement pu être consacrée à l'ajout de fonctionnalités supplémentaires.

Certaines informations utiles n'ont également été découvertes qu'en cours ou en fin de projet. A titre d'exemple nous avons appris tardivement l'existence de certaines ressources documentaires qui auraient pu faciliter notre compréhension de l'architecture et de l'historique des choix techniques réalisés.

Enfin, l'organisation des échanges a parfois été difficile, dû aux contraintes de disponibilité des différents acteurs du projet. La coordination entre les membres de l'équipe et les encadrants nécessite de concilier plusieurs emplois du temps, ce qui peut ralentir certaines prises de décision ou validations techniques. De plus, nous avons majoritairement travaillé au bâtiment Michel Dubois et non au laboratoire, ce qui réduit les opportunités de rencontres spontanées avec les membres de l'équipe et rend les échanges informels bien plus rares.

Malgré ces difficultés, nous avons su adapter notre démarche tout au long du projet. Face à la complexité du code existant, nous avons progressivement réévalué nos priorités afin de concentrer nos efforts sur les éléments ayant le plus fort impact pour la pérennité du projet. Nous avons été rigoureuses sur la refonte du code et avons mis l'accent sur la documentation et la maintenance du projet, plutôt qu'à l'ajout de nouvelles fonctionnalités. Par manque de temps, nous avons notamment renoncé à certaines tâches initialement envisagées, comme la mise à jour des scrapers*, ayant tenu à fournir un résultat de qualité afin de faciliter le travail des futurs contributeurs.

Améliorations possibles

Nous avons pu identifier plusieurs pistes d'amélioration possibles.

Tout d'abord, le développement d'un outil de scraping* adaptatif constitue une piste d'évolution importante, comme ce que nous avons initialement prévu au début du TER. Une maintenance régulière des scrapers* et des parsers* reste nécessaire afin de s'adapter aux évolutions fréquentes des sites de presse. Dans cette optique, l'utilisation de bibliothèques spécialisées dans le scraping* adaptatif pourrait être étudiée. Au cours de nos recherches,

nous avons notamment identifié la bibliothèque *Scrapling*, qui propose des mécanismes visant à rendre le scraping* plus robuste face aux changements de structure des pages web.

Par ailleurs, l'intelligence artificielle de catégorisation des articles pourrait être améliorée. Ce composant reste aujourd'hui relativement isolé du reste de l'architecture et ne bénéficie pas des mêmes principes d'organisation que les modules ayant fait l'objet de la refonte. Une évolution de ce système permettrait de mieux l'intégrer à l'architecture globale du projet.

Enfin, les requêtes existantes parcourent la base entière à chaque opération et pourraient être optimisées pour ne plus avoir à le faire. L'ajout d'un champ `processed` possédant une valeur `true` ou `false` pourrait contribuer à améliorer les performances globales du système, en particulier lors des opérations de traitement sur de grands volumes de données. Les performances actuelles restent acceptables, mais d'ici plusieurs années, l'augmentation du nombre d'articles stockés pourrait être nuisible au projet.

Perspectives d'évolution du projet

Le travail réalisé durant ce TER constitue avant tout une base de travail pour les futurs contributeurs du projet Gendered News. La refonte de l'architecture, la clarification des responsabilités des modules ainsi que le développement d'une documentation solide devraient faciliter la compréhension du code et l'ajout de nouvelles fonctionnalités dans les années à venir. L'un des objectifs à long terme du projet étant de pouvoir intégrer de nouveaux processings* et indicateurs d'analyse, cette nouvelle organisation logicielle est pensée pour rendre ces évolutions plus simples à mettre en œuvre.

Enfin, notre travail et les discussions menées avec l'équipe projet ont mis en évidence que certaines limites actuelles sont également liées aux ressources disponibles. Une allocation plus importante de ressources techniques ou humaines pourrait permettre d'accélérer les évolutions futures, d'améliorer les performances des traitements et de renforcer la pérennité du projet sur le long terme.

Conclusion

Ce TER nous a permis de contribuer à l'évolution du projet Gendered News en nous concentrant principalement sur sa maintenabilité. Au cours de ces plusieurs mois, nous avons réalisé une analyse approfondie de l'existant, conçu puis implémenté une nouvelle architecture logicielle*, produit une documentation technique et mis en place des bases plus solides pour les futurs développeurs, tout en partageant nos connaissances avec l'équipe du projet.

Au-delà des réalisations techniques, cette expérience nous a confrontées à des problématiques fréquemment rencontrées dans un contexte professionnel : la compréhension d'un projet existant, la gestion de dette technique*, l'adaptation à des contraintes de temps et de ressources, ainsi que la collaboration avec différents acteurs du projet. Elle nous a également amenées à prendre des décisions de priorisation afin de concentrer nos efforts sur les éléments présentant le plus fort impact pour l'avenir du système.

Nous avons également pu renforcer nos compétences en conception logicielle, en développement Python, en documentation technique et en gestion de projet, ainsi que notre capacité à analyser une architecture de code complexe, à remettre en question certaines décisions techniques et à proposer des solutions adaptées aux contraintes rencontrées.

De plus, le projet nous a offert l'opportunité d'observer certaines différences entre le fonctionnement d'une entreprise et celui d'un laboratoire de recherche. Nous avons pu constater, par exemple, qu'un laboratoire dispose généralement de processus moins formalisés, avec moins d'outils et de protocoles imposés qu'en entreprise; ou que les emplois du temps plus variés des différents membres peuvent rendre les échanges et la coordination plus complexes, ce qui nécessite davantage d'autonomie et d'adaptation dans l'organisation du travail. Après avoir évolué à la fois dans un contexte académique de recherche et dans un cadre d'entreprise suite à nos expériences passées, nous disposons désormais d'une vision plus concrète des spécificités de chacun, et cette comparaison nous aidera à mieux orienter nos choix professionnels futurs en fonction de nos attentes et de notre manière de travailler.

Glossaire

Terme	Définition
Architecture logicielle	Structure d'un logiciel décrivant l'organisation de ses composants, leurs rôles et la manière dont ils communiquent entre eux.
Dettes techniques	Ensemble des imperfections d'un logiciel résultant de choix techniques passés, causant souvent de nombreux problèmes par la suite.
Git	Système de contrôle de version qui permet à une équipe de développeurs collaborateurs d'un projet de gérer les changements apportés au code.
Intégration continue (CI)	Pratique consistant à exécuter automatiquement des vérifications, comme des tests, à chaque modification du code afin de détecter rapidement les erreurs.
Parser	Outil permettant d'effectuer l'extraction des informations via le processus de parsing
Parsing	Processus consistant à analyser des données brutes (texte, page web, fichier, etc.) afin d'en extraire des informations exploitables par un programme.
Processing	Ensemble des traitements appliqués à un article après son extraction afin de l'enrichir ou d'en analyser le contenu avant son stockage.
Scraper	Outils permettant de récupérer le contenu d'une page HTML
Scraping	Processus automatisé permettant de récupérer le contenu de pages web afin d'en extraire des informations, comme des articles de presse, qui seront ensuite traitées par le système.

Bibliographie

[1] Gendered News Dashboard : <https://genderednews.univ-grenoble-alpes.fr/dashe/>

[2] Pdoc Documentation : <https://pdoc.dev/>

Résumé

Ce Travail d'Études et de Recherche s'est déroulé au sein de l'équipe GETALP du Laboratoire d'Informatique de Grenoble (LIG), dans le cadre du projet Gendered News. Ce projet vise à analyser la représentation des genres dans la presse française à travers la collecte et le traitement automatisé d'articles issus de différents médias en ligne.

Les principaux objectifs de ce TER ont été d'améliorer la maintenabilité du système de collecte d'articles, de réduire la dette technique* accumulée au fil des années et de faciliter les futures évolutions du projet. Pour répondre à ces enjeux, nous avons réalisé une analyse de l'architecture existante, puis nous avons conçu et implémenté une nouvelle organisation logicielle fondée sur une séparation des responsabilités entre les différents modules du système.

Parmi les travaux réalisés, il y a la refonte de l'architecture des modules de scraping*, de parsing* et de traitement des articles. Nous avons ajouté des tests unitaires automatisés ainsi que la création d'une documentation technique générée automatiquement à l'aide de la bibliothèque Pdoc. Nous avons également travaillé à l'amélioration du processus d'intégration continue afin de sécuriser les futurs développements.

Ce projet nous a permis de renforcer nos compétences en développement Python, en conception logicielle, en qualité logicielle et en documentation technique. Il nous a également sensibilisées aux problématiques de reprise d'un projet existant et de la gestion de sa dette technique*.

Mots-clés :

Python, scraping, parsing*, architecture logicielle, dette technique*, tests unitaires, intégration continue, documentation technique, maintenabilité

Annexe

Figure 1 - Diagramme de l'ancienne architecture logicielle

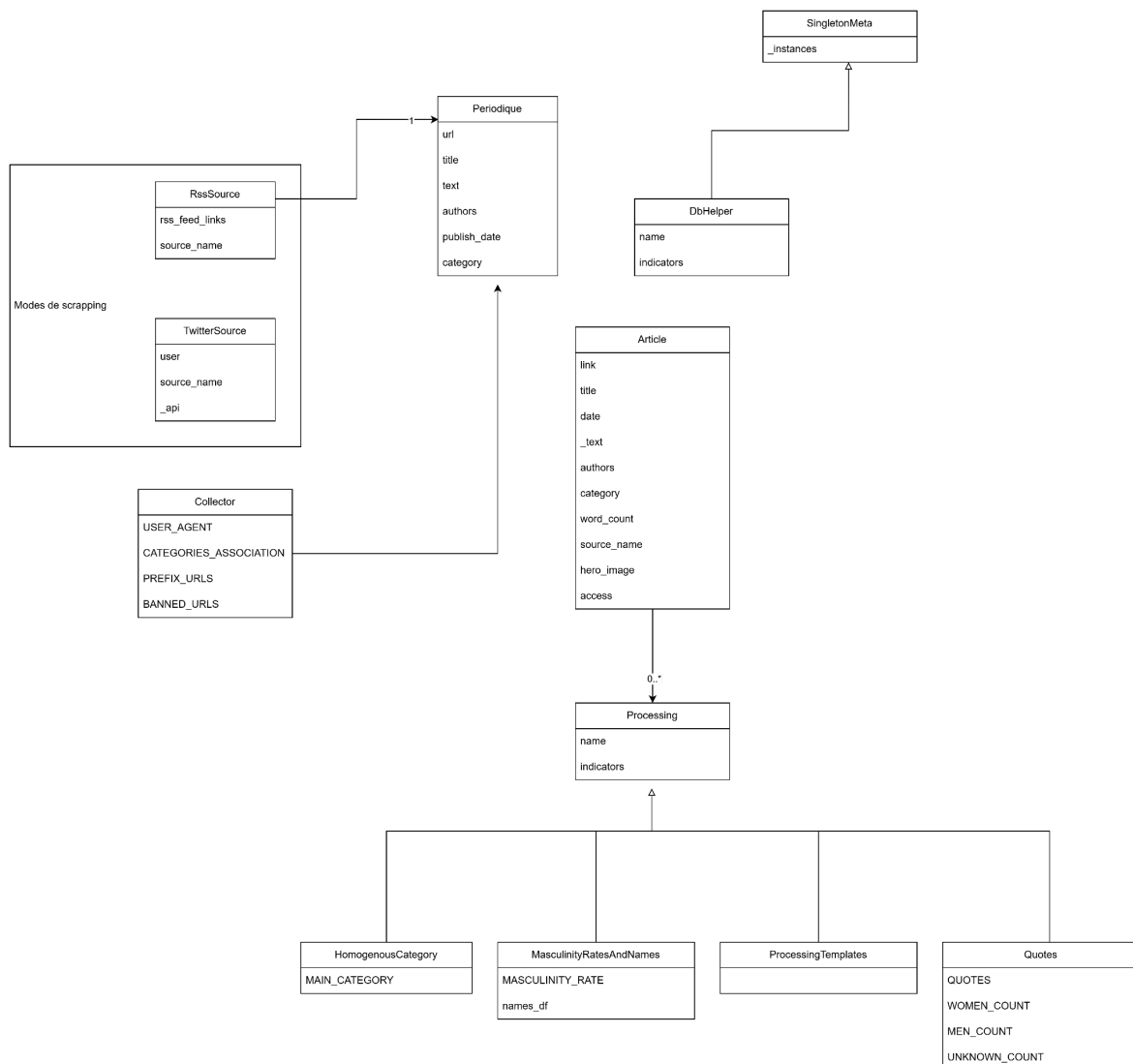


Figure 2 - Diagramme de la nouvelle architecture logicielle

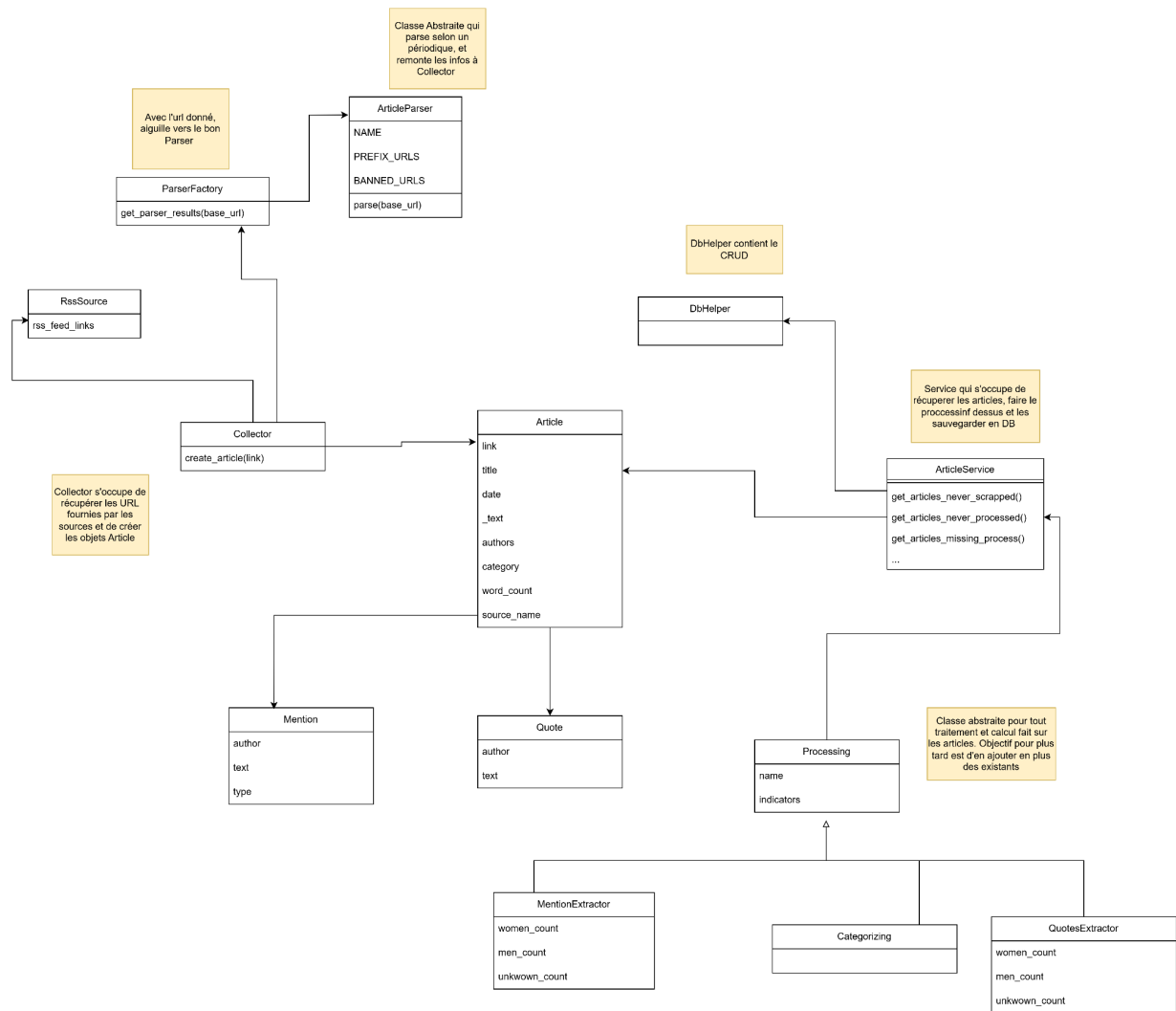


Figure 3 - Flux des étapes de la récupération d'articles

Configuration RSS

↓

Renvoi des URLs des flux RSS

↓

Parsing via ArticleParser

↓

Collector crée les objets Article

↓

Récupération des articles par le ArticleService

↓

Application des processings

↓

Envoi en base de données via le ArticleService

Figure 4 - Diagramme de Gantt

